HT:2403A54057

LOAD THE DATASET

```
import pandas as pd

df = pd.read_csv('/content/laptop_prices (1) (1).csv')

print("First 5 rows of the dataset:")
display(df.head())
```

```
First 5 rows of the dataset:
```

	Company	Product	TypeName	Inches	Ram	OS	Weight	Price_euros	Screen	Sc
0	Apple	MacBook Pro	Ultrabook	13.3	8	macOS	1.37	1339.69	Standard	
1	Apple	Macbook Air	Ultrabook	13.3	8	macOS	1.34	898.94	Standard	
2	HP	250 G6	Notebook	15.6	8	No OS	1.86	575.00	Full HD	
3	Apple	MacBook Pro	Ultrabook	15.4	16	macOS	1.83	2537.45	Standard	
4	Apple	MacBook Pro	Ultrabook	13.3	8	macOS	1.37	1803.60	Standard	

5 rows × 23 columns

DATA UNDERSTANDING

```
# Identify categorical columns for one-hot encoding (excluding high cardinality one
categorical_cols = [
    'Company',
    'TypeName',
    'OS',
    'Screen',
    'Touchscreen',
    'IPSpanel',
    'RetinaDisplay',
    'CPU_company',
    'PrimaryStorageType',
    'SecondaryStorageType',
    'GPU_company'
]

# Apply one-hot encoding
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```

```
print("First 5 rows of the DataFrame after one-hot encoding:")
display(df_encoded.head())
print(f"\nShape of the DataFrame after one-hot encoding: {df_encoded.shape}")
```

```
First 5 rows of the DataFrame after one-hot encoding:
```

	Product	Inches	Ram	Weight	Price_euros	ScreenW	ScreenH	CPU_freq	CPU_model
0	MacBook Pro	13.3	8	1.37	1339.69	2560	1600	2.3	Core i5
1	Macbook Air	13.3	8	1.34	898.94	1440	900	1.8	Core i5
2	250 G6	15.6	8	1.86	575.00	1920	1080	2.5	Core i5 7200U
3	MacBook Pro	15.4	16	1.83	2537.45	2880	1800	2.7	Core i7
4	MacBook Pro	13.3	8	1.37	1803.60	2560	1600	3.1	Core i5

5 rows × 60 columns

```
Shape of the DataFrame after one-hot encoding: (1275, 60)
```

```
# Identify categorical columns for one-hot encoding (excluding high cardinality one
categorical_cols = [
    'Company',
    'TypeName',
    'OS',
    'Screen',
    'Touchscreen',
    'IPSpanel',
    'RetinaDisplay',
    'CPU_company',
    'PrimaryStorageType',
    'SecondaryStorageType',
    'GPU_company'
]

# Apply one-hot encoding
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

print("First 5 rows of the DataFrame after one-hot encoding:")
display(df_encoded.head())
print(f"\nShape of the DataFrame after one-hot encoding: {df_encoded.shape}")
```

First 5 rows of the DataFrame after one-hot encoding:

	Product	Inches	Ram	Weight	Price_euros	ScreenW	ScreenH	CPU_freq	CPU_model
0	MacBook Pro	13.3	8	1.37	1339.69	2560	1600	2.3	Core i5
1	Macbook Air	13.3	8	1.34	898.94	1440	900	1.8	Core i5
2	250 G6	15.6	8	1.86	575.00	1920	1080	2.5	Core i5 7200U
3	MacBook Pro	15.4	16	1.83	2537.45	2880	1800	2.7	Core i7
4	MacBook Pro	13.3	8	1.37	1803.60	2560	1600	3.1	Core i5

5 rows × 60 columns

```
Shape of the DataFrame after one-hot encoding: (1275, 60)
```

```
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1275 entries, 0 to 1274
Data columns (total 23 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   Company               1275 non-null   object 
 1   Product               1275 non-null   object 
 2   TypeName              1275 non-null   object 
 3   Inches                1275 non-null   float64
 4   Ram                   1275 non-null   int64  
 5   OS                    1275 non-null   object 
 6   Weight                1275 non-null   float64
 7   Price_euros           1275 non-null   float64
 8   Screen                1275 non-null   object 
 9   ScreenW               1275 non-null   int64  
 10  ScreenH               1275 non-null   int64  
 11  Touchscreen           1275 non-null   object 
 12  IPSpanel              1275 non-null   object 
 13  RetinaDisplay         1275 non-null   object 
 14  CPU_company           1275 non-null   object 
 15  CPU_freq              1275 non-null   float64
 16  CPU_model             1275 non-null   object 
 17  PrimaryStorage        1275 non-null   int64  
 18  SecondaryStorage      1275 non-null   int64  
 19  PrimaryStorageType    1275 non-null   object 
 20  SecondaryStorageType  1275 non-null   object 
 21  GPU_company           1275 non-null   object 
 22  GPU_model             1275 non-null   object 
dtypes: float64(4), int64(5), object(14)
memory usage: 229.2+ KB
```

```
print("\nDescriptive Statistics:")
display(df.describe(include='all'))
```

Descriptive Statistics:

	Company	Product	TypeName	Inches	Ram	OS	Weight	P
count	1275	1275	1275	1275.000000	1275.000000	1275	1275.000000	1
unique	19	618	6	NaN	NaN	9	NaN	
top	Dell	XPS 13	Notebook	NaN	NaN	Windows 10	NaN	
freq	291	30	707	NaN	NaN	1048	NaN	
mean	NaN	NaN	NaN	15.022902	8.440784	NaN	2.040525	1
std	NaN	NaN	NaN	1.429470	5.097809	NaN	0.669196	
min	NaN	NaN	NaN	10.100000	2.000000	NaN	0.690000	
25%	NaN	NaN	NaN	14.000000	4.000000	NaN	1.500000	
50%	NaN	NaN	NaN	15.600000	8.000000	NaN	2.040000	
75%	NaN	NaN	NaN	15.600000	8.000000	NaN	2.310000	1
max	NaN	NaN	NaN	18.400000	64.000000	NaN	4.700000	6

11 rows × 23 columns

```
print("\nMissing Values:")
display(df.isnull().sum())
```

```
Missing Values:
```

	0
Company	0
Product	0
TypeName	0
Inches	0
Ram	0
OS	0
Weight	0
Price_euros	0
Screen	0
ScreenW	0
ScreenH	0
Touchscreen	0
IPSpanel	0
RetinaDisplay	0
CPU_company	0
CPU_freq	0
CPU_model	0
PrimaryStorage	0
SecondaryStorage	0
PrimaryStorageType	0
SecondaryStorageType	0
GPU_company	0
GPU_model	0

dtype: int64

```
from sklearn.model_selection import train_test_split

# Define features (X) and target (y)
# We'll use the one-hot encoded DataFrame df_encoded for features
# and 'Price_euros' as the target variable

X = df_encoded.drop('Price_euros', axis=1)
y = df_encoded['Price_euros']

# Split the data into training and testing sets (e.g., 80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

print("Shape of X_train:", X_train.shape)
```

```
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)
```

```
Shape of X_train: (1020, 59)
Shape of X_test: (255, 59)
Shape of y_train: (1020,)
Shape of y_test: (255,)
```

```
from sklearn.preprocessing import StandardScaler

# Identify numerical columns for scaling
# Exclude 'Product', 'CPU_model', 'GPU_model' as they are high-cardinality object t
# Exclude one-hot encoded columns (they are binary and don't need scaling in the sa

# Get numerical columns from X_train (excluding object type columns)
numerical_cols = X_train.select_dtypes(include=['int64', 'float64']).columns

# Initialize the StandardScaler
scaler = StandardScaler()

# Apply scaling to the numerical columns in X_train
X_train[numerical_cols] = scaler.fit_transform(X_train[numerical_cols])

# Apply scaling to the numerical columns in X_test using the SAME scaler fitted on
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])

print("First 5 rows of X_train after standardization:")
display(X_train.head())
```

First 5 rows of X_train after standardization:

	Product	Inches	Ram	Weight	ScreenW	ScreenH	CPU_freq	CPU_model
413	Aspire R7	-1.194933	-0.060972	-0.651198	0.038594	0.020878	0.403751	Core i7 6500U
778	Blade Pro	-0.704288	1.513485	-0.119813	0.038594	0.020878	0.996591	Core i7 7700HQ
1107	Yoga 500-15ISK	0.417185	-0.848200	0.107923	0.038594	0.020878	0.008524	Core i5 6200U
96	Inspiron 3567	0.417185	-0.060972	0.259747	0.038594	0.020878	0.798977	Core i7 7500U
309	250 G6	0.417185	-0.848200	-0.256455	0.038594	0.020878	-0.584315	Core i3 6006U

5 rows × 59 columns

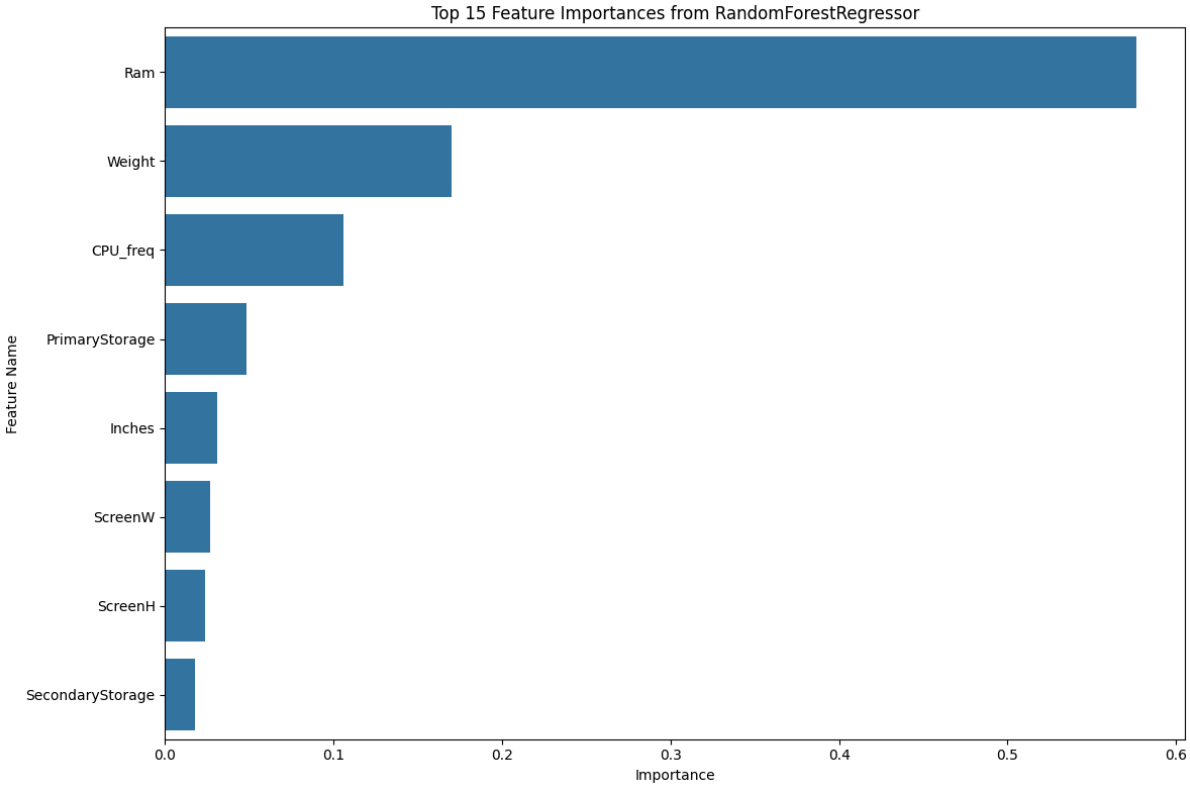
```
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns # Import seaborn

# Initialize and train a RandomForestRegressor model
# Using random_state for reproducibility
model = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
model.fit(X_train.select_dtypes(include=np.number), y_train) # Fit only on numerica

# Get feature importances
feature_importances = model.feature_importances_

# Get feature names
feature_names = X_train.select_dtypes(include=np.number).columns

# Create a pandas Series for easy viewing and sorting
importance_df = pd.Series(feature_importances, index=feature_names).sort_values(asc

print("Top 15 Feature Importances:")
display(importance_df.head(15))

# Plotting feature importances
plt.figure(figsize=(12, 8))
sns.barplot(x=importance_df.head(15).values, y=importance_df.head(15).index)
plt.title('Top 15 Feature Importances from RandomForestRegressor')
plt.xlabel('Importance')
plt.ylabel('Feature Name')
plt.tight_layout()
plt.show()
```

```
Top 15 Feature Importances:
```

	0
Ram	0.576490
Weight	0.169807
CPU_freq	0.105742
PrimaryStorage	0.048548
Inches	0.030812
ScreenW	0.026941
ScreenH	0.023663
SecondaryStorage	0.017997

dtype: float64




```
print("\nX_train Data Types:")
X_train.info()
```

```
 3   Weight                      1020 non-null   float64
 4   ScreenW                     1020 non-null   float64
 5   ScreenH                     1020 non-null   float64
```

```
 16  Company_Google              1020 non-null   bool
 17  Company_HP                  1020 non-null   bool
 18  Company_Huawei              1020 non-null   bool
 19  Company_LG                  1020 non-null   bool
 20  Company_Lenovo              1020 non-null   bool
 21  Company_MSI                 1020 non-null   bool
 22  Company_Mediacom            1020 non-null   bool
 23  Company_Microsoft           1020 non-null   bool
 24  Company_Razer               1020 non-null   bool
 25  Company_Samsung             1020 non-null   bool
 26  Company_Toshiba             1020 non-null   bool
 27  Company_Vero                1020 non-null   bool
 28  Company_Xiaomi              1020 non-null   bool
 29  TypeName_Gaming             1020 non-null   bool
 30  TypeName_Netbook            1020 non-null   bool
 31  TypeName_Notebook           1020 non-null   bool
 32  TypeName_Ultrabook          1020 non-null   bool
 33  TypeName_Workstation        1020 non-null   bool
 34  OS_Chrome OS                1020 non-null   bool
 35  OS_Linux                    1020 non-null   bool
 36  OS_Mac OS X                 1020 non-null   bool
 37  OS_No OS                    1020 non-null   bool
 38  OS_Windows 10               1020 non-null   bool
 39  OS_Windows 10 S             1020 non-null   bool
 40  OS_Windows 7                1020 non-null   bool
 41  OS_macOS                    1020 non-null   bool
 42  Screen_Full HD              1020 non-null   bool
 43  Screen_Quad HD+             1020 non-null   bool
 44  Screen_Standard             1020 non-null   bool
 45  Touchscreen_Yes             1020 non-null   bool
 46  IPSpanel_Yes                1020 non-null   bool
 47  RetinaDisplay_Yes           1020 non-null   bool
 48  CPU_company_Intel           1020 non-null   bool
 49  CPU_company_Samsung         1020 non-null   bool
 50  PrimaryStorageType_HDD      1020 non-null   bool
 51  PrimaryStorageType_Hybrid   1020 non-null   bool
 52  PrimaryStorageType_SSD      1020 non-null   bool
 53  SecondaryStorageType_Hybrid 1020 non-null   bool
 54  SecondaryStorageType_No     1020 non-null   bool
 55  SecondaryStorageType_SSD    1020 non-null   bool
 56  GPU_company_ARM             1020 non-null   bool
 57  GPU_company_Intel           1020 non-null   bool
 58  GPU_company_Nvidia          1020 non-null   bool
dtypes: bool(48), float64(8), object(3)
memory usage: 143.4+ KB
```

```
print("\nX_test Data Types:")
X_test.info()
```

```
 3   Weight                      255 non-null    float64
 4   ScreenW                     255 non-null    float64
 5   ScreenH                     255 non-null    float64
 6   CPU_freq                    255 non-null    float64
 7   CPU_model                   255 non-null    object
```

```
 18  Company_Huawei                255 non-null    bool
 19  Company_LG                    255 non-null    bool
 20  Company_Lenovo                255 non-null    bool
 21  Company_MSI                   255 non-null    bool
 22  Company_Mediacom              255 non-null    bool
 23  Company_Microsoft             255 non-null    bool
 24  Company_Razer                 255 non-null    bool
 25  Company_Samsung               255 non-null    bool
 26  Company_Toshiba               255 non-null    bool
 27  Company_Vero                  255 non-null    bool
 28  Company_Xiaomi                255 non-null    bool
 29  TypeName_Gaming               255 non-null    bool
 30  TypeName_Netbook              255 non-null    bool
 31  TypeName_Notebook             255 non-null    bool
 32  TypeName_Ultrabook            255 non-null    bool
 33  TypeName_Workstation          255 non-null    bool
 34  OS_Chrome OS                  255 non-null    bool
 35  OS_Linux                      255 non-null    bool
 36  OS_Mac OS X                   255 non-null    bool
 37  OS_No OS                      255 non-null    bool
 38  OS_Windows 10                 255 non-null    bool
 39  OS_Windows 10 S               255 non-null    bool
 40  OS_Windows 7                  255 non-null    bool
 41  OS_macOS                      255 non-null    bool
 42  Screen_Full HD                255 non-null    bool
 43  Screen_Quad HD+               255 non-null    bool
 44  Screen_Standard               255 non-null    bool
 45  Touchscreen_Yes               255 non-null    bool
 46  IPSpanel_Yes                  255 non-null    bool
 47  RetinaDisplay_Yes             255 non-null    bool
 48  CPU_company_Intel             255 non-null    bool
 49  CPU_company_Samsung           255 non-null    bool
 50  PrimaryStorageType_HDD        255 non-null    bool
 51  PrimaryStorageType_Hybrid     255 non-null    bool
 52  PrimaryStorageType_SSD        255 non-null    bool
 53  SecondaryStorageType_Hybrid   255 non-null    bool
 54  SecondaryStorageType_No       255 non-null    bool
 55  SecondaryStorageType_SSD      255 non-null    bool
 56  GPU_company_ARM               255 non-null    bool
 57  GPU_company_Intel             255 non-null    bool
 58  GPU_company_Nvidia            255 non-null    bool
dtypes: bool(48), float64(8), object(3)
memory usage: 35.9+ KB
```

```
# Drop the remaining object columns from X_train and X_test
high_cardinality_cols = ['Product', 'CPU_model', 'GPU_model']

X_train = X_train.drop(columns=high_cardinality_cols)
X_test = X_test.drop(columns=high_cardinality_cols)

print("\nX_train Data Types after dropping object columns:")
X_train.info()

print("\nX_test Data Types after dropping object columns:")
X_test.info()
```

```
 8   Company_Apple                255 non-null    bool
 9   Company_Asus                 255 non-null    bool
 10  Company_Chuwi                255 non-null    bool
 11  Company_Dell                 255 non-null    bool
 12  Company_Fujitsu              255 non-null    bool
 13  Company_Google               255 non-null    bool
 14  Company_HP                   255 non-null    bool
 15  Company_Huawei               255 non-null    bool
 16  Company_LG                   255 non-null    bool
 17  Company_Lenovo               255 non-null    bool
 18  Company_MSI                  255 non-null    bool
 19  Company_Mediacom             255 non-null    bool
 20  Company_Microsoft            255 non-null    bool
 21  Company_Razer                255 non-null    bool
 22  Company_Samsung              255 non-null    bool
 23  Company_Toshiba              255 non-null    bool
 24  Company_Vero                 255 non-null    bool
 25  Company_Xiaomi               255 non-null    bool
 26  TypeName_Gaming              255 non-null    bool
 27  TypeName_Netbook             255 non-null    bool
 28  TypeName_Notebook            255 non-null    bool
 29  TypeName_Ultrabook           255 non-null    bool
 30  TypeName_Workstation         255 non-null    bool
 31  OS_Chrome OS                 255 non-null    bool
 32  OS_Linux                     255 non-null    bool
 33  OS_Mac OS X                  255 non-null    bool
 34  OS_No OS                     255 non-null    bool
 35  OS_Windows 10                255 non-null    bool
 36  OS_Windows 10 S              255 non-null    bool
 37  OS_Windows 7                 255 non-null    bool
 38  OS_macOS                     255 non-null    bool
 39  Screen_Full HD               255 non-null    bool
 40  Screen_Quad HD+              255 non-null    bool
 41  Screen_Standard              255 non-null    bool
 42  Touchscreen_Yes              255 non-null    bool
 43  IPSpanel_Yes                 255 non-null    bool
 44  RetinaDisplay_Yes            255 non-null    bool
 45  CPU_company_Intel            255 non-null    bool
 46  CPU_company_Samsung          255 non-null    bool
 47  PrimaryStorageType_HDD       255 non-null    bool
 48  PrimaryStorageType_Hybrid    255 non-null    bool
 49  PrimaryStorageType_SSD       255 non-null    bool
 50  SecondaryStorageType_Hybrid  255 non-null    bool
 51  SecondaryStorageType_No      255 non-null    bool
 52  SecondaryStorageType_SSD     255 non-null    bool
 53  GPU_company_ARM              255 non-null    bool
 54  GPU_company_Intel            255 non-null    bool
 55  GPU_company_Nvidia           255 non-null    bool
dtypes: bool(48), float64(8)
memory usage: 29.9 KB
```

```
from sklearn.ensemble import RandomForestRegressor

# Initialize the Random Forest Regressor model with a random state for reproducibil
model = RandomForestRegressor(n_estimators=100, random_state=42)

# Train the model using the training data
model.fit(X_train, y_train)

print("Random Forest Regressor model trained successfully!")
```

```
Random Forest Regressor model trained successfully!
```

```
from sklearn.linear_model import LinearRegression

# Initialize the Linear Regression model
linear_model = LinearRegression()

# Train the model using the training data
linear_model.fit(X_train, y_train)

print("Linear Regression model trained successfully!")
```

```
Linear Regression model trained successfully!
```

```
from sklearn.tree import DecisionTreeRegressor

# Initialize the Decision Tree Regressor model
decision_tree_model = DecisionTreeRegressor(random_state=42)

# Train the model using the training data
decision_tree_model.fit(X_train, y_train)

print("Decision Tree Regressor model trained successfully!")
```

```
Decision Tree Regressor model trained successfully!
```

```
from sklearn.svm import SVR

# Initialize the Support Vector Regressor model
# Using a linear kernel for simplicity as a start, can be tuned later
svr_model = SVR(kernel='linear')

# Train the model using the training data
svr_model.fit(X_train, y_train)

print("Support Vector Regressor model trained successfully!")
```

```
Support Vector Regressor model trained successfully!
```

```
Start coding or generate with AI.
```

```
from sklearn.neighbors import KNeighborsRegressor

# Initialize the K-Nearest Neighbors Regressor model
# Using n_neighbors=5 as a starting point, can be tuned later
knn_model = KNeighborsRegressor(n_neighbors=5)

# Train the model using the training data
knn_model.fit(X_train, y_train)

print("K-Nearest Neighbors Regressor model trained successfully!")
```

```
K-Nearest Neighbors Regressor model trained successfully!
```

```
from sklearn.ensemble import GradientBoostingRegressor

# Initialize the Gradient Boosting Regressor model
gbr_model = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_dept
```

```
# Train the model using the training data
gbr_model.fit(X_train, y_train)

print("Gradient Boosting Regressor model trained successfully!")
```

```
Gradient Boosting Regressor model trained successfully!
```

```
from sklearn.ensemble import AdaBoostRegressor

# Initialize the AdaBoost Regressor model
adaboost_model = AdaBoostRegressor(n_estimators=100, random_state=42)

# Train the model using the training data
adaboost_model.fit(X_train, y_train)

print("AdaBoost Regressor model trained successfully!")
```

```
AdaBoost Regressor model trained successfully!
```

```
import pandas as pd
from sklearn.model_selection import KFold, cross_val_score, train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
import numpy as np

# --- Re-running necessary data loading and preprocessing steps to ensure X_train a

# Load the dataset (assuming it's in the same path as before)
df = pd.read_csv('/content/laptop_prices (1) (1).csv')

# Identify categorical columns for one-hot encoding (excluding high cardinality one
categorical_cols = [
    'Company',
    'TypeName',
    'OS',
    'Screen',
    'Touchscreen',
    'IPSpanel',
    'RetinaDisplay',
    'CPU_company',
    'PrimaryStorageType',
    'SecondaryStorageType',
    'GPU_company'
]

# Apply one-hot encoding
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Define features (X) and target (y)
X = df_encoded.drop('Price_euros', axis=1)
y = df_encoded['Price_euros']

# Split the data into training and testing sets (e.g., 80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Identify numerical columns for scaling
numerical_cols = X_train.select_dtypes(include=['int64', 'float64']).columns
```

```
# Initialize the StandardScaler
scaler = StandardScaler()

# Apply scaling to the numerical columns in X_train
X_train[numerical_cols] = scaler.fit_transform(X_train[numerical_cols])

# Apply scaling to the numerical columns in X_test using the SAME scaler fitted on
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])

# Drop the remaining object columns from X_train and X_test
high_cardinality_cols = ['Product', 'CPU_model', 'GPU_model']
X_train = X_train.drop(columns=high_cardinality_cols)
X_test = X_test.drop(columns=high_cardinality_cols)

# --- End of re-running preprocessing ---

# Initialize the Random Forest Regressor model with a random state for reproducibil
model = RandomForestRegressor(n_estimators=100, random_state=42)

# Train the model using the training data
model.fit(X_train, y_train)

# Define the number of folds for cross-validation
n_splits = 5
kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)

# Perform cross-validation on the Random Forest Regressor model
cv_scores = cross_val_score(model, X_train, y_train, cv=kf, scoring='neg_mean_squar

# Convert negative MSE to positive RMSE for better interpretability
rmse_scores = np.sqrt(-cv_scores)

print(f"Cross-validation RMSE scores: {rmse_scores}")
print(f"Mean RMSE: {np.mean(rmse_scores):.2f}")
print(f"Standard deviation of RMSE: {np.std(rmse_scores):.2f}")
```

```
Cross-validation RMSE scores: [366.32666266 293.32749963 306.01530824 268.74788451 3
Mean RMSE: 307.16
Standard deviation of RMSE: 32.26
```

```
from sklearn.decomposition import PCA

# Initialize PCA to retain 95% of the variance
pca = PCA(n_components=0.95, random_state=42)

# Fit PCA on the training data and transform both training and test data
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

print(f"Original number of features: {X_train.shape[1]}")
print(f"Number of features after PCA (X_train): {X_train_pca.shape[1]}")
print(f"Shape of X_train_pca: {X_train_pca.shape}")
print(f"Shape of X_test_pca: {X_test_pca.shape}")

print(f"Variance explained by selected components: {pca.explained_variance_ratio_.s
```

```
Original number of features: 56
Number of features after PCA (X_train): 17
Shape of X_train_pca: (1020, 17)
Shape of X_test_pca: (255, 17)
Variance explained by selected components: 0.95
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import GradientBoostingRegressor, AdaBoostRegressor
import pandas as pd
import numpy as np

# Dictionary to store models and their names
models = {
    'RandomForestRegressor': RandomForestRegressor(n_estimators=100, random_state=4
    'LinearRegression': LinearRegression(),
    'DecisionTreeRegressor': DecisionTreeRegressor(random_state=42),
    'SVR': SVR(kernel='linear'),
    'KNeighborsRegressor': KNeighborsRegressor(n_neighbors=5),
    'GradientBoostingRegressor': GradientBoostingRegressor(n_estimators=100, learni
    'AdaBoostRegressor': AdaBoostRegressor(n_estimators=100, random_state=42)
}

# DataFrame to store results
results_pca = []

for name, model in models.items():
    print(f"Training {name}...")
    # Train the model on PCA-transformed data
    model.fit(X_train_pca, y_train)

    # Make predictions on PCA-transformed test data
    y_pred_pca = model.predict(X_test_pca)

    # Evaluate the model
    mae = mean_absolute_error(y_test, y_pred_pca)
    mse = mean_squared_error(y_test, y_pred_pca)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred_pca)

    results_pca.append({
        'Model': name,
        'MAE': mae,
        'MSE': mse,
        'RMSE': rmse,
        'R2 Score': r2
    })

# Convert results to a DataFrame
df_results_pca = pd.DataFrame(results_pca)

print("\nModel Comparison Results on PCA-transformed Data:")
display(df_results_pca.sort_values(by='RMSE'))
```

```
Training RandomForestRegressor...
Training LinearRegression...
Training DecisionTreeRegressor...
Training SVR...
Training KNeighborsRegressor...
Training GradientBoostingRegressor...
Training AdaBoostRegressor...

Model Comparison Results on PCA-transformed Data:
```

	Model	MAE	MSE	RMSE	R2 Score
5	GradientBoostingRegressor	237.956736	100946.803983	317.721268	0.796617
0	RandomForestRegressor	230.432227	107360.980552	327.659855	0.783694
4	KNeighborsRegressor	258.532706	134951.818166	367.357888	0.728106
1	LinearRegression	280.365518	139078.961729	372.932919	0.719791
3	SVR	303.046843	187373.845271	432.867006	0.622488
2	DecisionTreeRegressor	306.789634	215882.558396	464.631637	0.565050
6	AdaBoostRegressor	400.524572	222807.429633	472.024819	0.551099

```
from sklearn.metrics import roc_curve, mean_absolute_error, mean_squared_error, r2_s
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, AdaBo
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.decomposition import PCA # Import PCA
import numpy as np
import pandas as pd # Import pandas
from sklearn.model_selection import train_test_split # Import train_test_split
from sklearn.preprocessing import StandardScaler # Import StandardScaler

print("Attempting to calculate ROC curve for a regression problem...")

# --- Re-running necessary data loading and preprocessing steps to ensure X_train an

# Load the dataset (assuming it's in the same path as before)
df = pd.read_csv('/content/laptop_prices (1) (1).csv')

# Identify categorical columns for one-hot encoding (excluding high cardinality ones
categorical_cols = [
    'Company',
    'TypeName',
    'OS',
    'Screen',
    'Touchscreen',
    'IPSpanel',
    'RetinaDisplay',
    'CPU_company',
    'PrimaryStorageType',
    'SecondaryStorageType',
    'GPU_company'
]

# Apply one-hot encoding
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```

```
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Define features (X) and target (y)
X = df_encoded.drop('Price_euros', axis=1)
y = df_encoded['Price_euros']

# Split the data into training and testing sets (e.g., 80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_stat

# Identify numerical columns for scaling
numerical_cols = X_train.select_dtypes(include=['int64', 'float64']).columns

# Initialize the StandardScaler
scaler = StandardScaler()

# Apply scaling to the numerical columns in X_train
X_train[numerical_cols] = scaler.fit_transform(X_train[numerical_cols])

# Apply scaling to the numerical columns in X_test using the SAME scaler fitted on X_
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])

# Drop the remaining object columns from X_train and X_test
high_cardinality_cols = ['Product', 'CPU_model', 'GPU_model']
X_train = X_train.drop(columns=high_cardinality_cols)
X_test = X_test.drop(columns=high_cardinality_cols)

# --- End of re-running preprocessing ---


# Redefine the models dictionary for this cell's scope
models = {
    'RandomForestRegressor': RandomForestRegressor(n_estimators=100, random_state=42
    'LinearRegression': LinearRegression(),
    'DecisionTreeRegressor': DecisionTreeRegressor(random_state=42),
    'SVR': SVR(kernel='linear'),
    'KNeighborsRegressor': KNeighborsRegressor(n_neighbors=5),
    'GradientBoostingRegressor': GradientBoostingRegressor(n_estimators=100, learning
    'AdaBoostRegressor': AdaBoostRegressor(n_estimators=100, random_state=42)
}

# --- Re-apply PCA transformation to ensure X_train_pca and X_test_pca are defined in
# Initialize PCA to retain 95% of the variance
pca = PCA(n_components=0.95, random_state=42)

# Fit PCA on the training data and transform both training and test data
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
# --- End of PCA re-application ---


# To make predictions, models need to be fitted on PCA-transformed data
for name, model in models.items():
    model.fit(X_train_pca, y_train)


try:
    # Pick one model's predictions, e.g., RandomForestRegressor
    rf_model = models['RandomForestRegressor']
    y_pred_rf_pca = rf_model.predict(X_test_pca)
```

```
    # This will raise a ValueError because `y_true` must be binary
    fpr, tpr, thresholds = roc_curve(y_test, y_pred_rf_pca)
    print("ROC curve calculated successfully. (Note: This output is unexpected for r
except ValueError as e:
    print(f"\nCaught expected error: {e}")
    print("This error demonstrates that `roc_curve` requires binary labels (0 or 1) 
    print("Since our target variable `Price_euros` is continuous, ROC curves are not


print("\nInstead of ROC curves, for regression, we focus on metrics like MAE, MSE, RI
```

```
calculate ROC curve for a regression problem...

ed error: continuous format is not supported
monstrates that `roc_curve` requires binary labels (0 or 1) for `y_true` and `y_score
get variable `Price_euros` is continuous, ROC curves are not appropriate for evaluati

 curves, for regression, we focus on metrics like MAE, MSE, RMSE, and R2 score.
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, AdaBo
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor

# --- Data Loading and Preprocessing Steps (Copied from earlier cells for self-conta
# Load the dataset
df = pd.read_csv('/content/laptop_prices (1) (1).csv')

# Identify categorical columns for one-hot encoding
categorical_cols = [
    'Company', 'TypeName', 'OS', 'Screen', 'Touchscreen', 'IPSpanel',
    'RetinaDisplay', 'CPU_company', 'PrimaryStorageType', 'SecondaryStorageType',
    'GPU_company'
]

# Apply one-hot encoding
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Define features (X) and target (y)
X = df_encoded.drop('Price_euros', axis=1)
y = df_encoded['Price_euros']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_stat

# Identify numerical columns for scaling
numerical_cols = X_train.select_dtypes(include=['int64', 'float64']).columns

# Initialize and apply StandardScaler
scaler = StandardScaler()
X_train[numerical_cols] = scaler.fit_transform(X_train[numerical_cols])
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])
```

```
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])

# Drop high-cardinality object columns
high_cardinality_cols = ['Product', 'CPU_model', 'GPU_model']
X_train = X_train.drop(columns=high_cardinality_cols)
X_test = X_test.drop(columns=high_cardinality_cols)

# --- PCA Transformation (Copied from earlier cells for self-containment) ---
pca = PCA(n_components=0.95, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

# --- Model Training and Evaluation ---
# Dictionary to store models and their names
models = {
    'RandomForestRegressor': RandomForestRegressor(n_estimators=100, random_state=42
    'LinearRegression': LinearRegression(),
    'DecisionTreeRegressor': DecisionTreeRegressor(random_state=42),
    'SVR': SVR(kernel='linear'),
    'KNeighborsRegressor': KNeighborsRegressor(n_neighbors=5),
    'GradientBoostingRegressor': GradientBoostingRegressor(n_estimators=100, learnin
    'AdaBoostRegressor': AdaBoostRegressor(n_estimators=100, random_state=42)
}

# DataFrame to store results
results_pca = []

for name, model in models.items():
    print(f"Training {name}...")
    # Train the model on PCA-transformed data
    model.fit(X_train_pca, y_train)

    # Make predictions on PCA-transformed test data
    y_pred_pca = model.predict(X_test_pca)

    # Evaluate the model
    mae = mean_absolute_error(y_test, y_pred_pca)
    mse = mean_squared_error(y_test, y_pred_pca)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred_pca)

    results_pca.append({
        'Model': name,
        'MAE': mae
```